

Homework Report

Jakub Nowacki

June 1, 2025

Introduction

I have completed all the tasks. The task was challenging mainly due to the GPU RAM required for computations. Managing all the necessary runtimes and shuffling files between them was the biggest problem for me. Unfortunately i could not keep all the logs in the notebook, because i was running it in parts on different runtimes and running it whole would take a lot of time.

1 Task 1

Generated Images

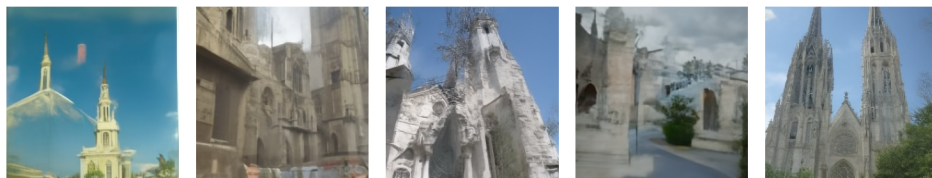


Figure 1: DDIM 10



Figure 2: DDIM 200



Figure 3: DDPM 10



Figure 4: DDPM 200

Findings

Answer:

The key difference between DDIM and DDPM lies in how they traverse the latent space during the denoising process, which can be understood from the formula for the update step:

$$x_{t-1} = \underbrace{\frac{\sqrt{\alpha_{t-1}}}{\sqrt{\alpha_t}} x_t - \frac{\sqrt{1 - \alpha_{t-1}}}{\sqrt{\alpha_t}} \epsilon_\theta(x_t)}_{\text{"predicted } x_0 \text{ term"}} + \underbrace{\sqrt{1 - \alpha_{t-1} - \sigma_t^2} \epsilon_\theta(x_t)}_{\text{"direction pointing to } x_t \text{ term"}} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}}$$

- **DDIM** is deterministic because it sets the noise variance term $\sigma_t = 0$, effectively removing the last noise term. This forces the sampling trajectory to follow a deterministic path through the latent space.
 - As a result, even timesteps set to 10, the generated images are already quite sharp because the model follows a consistent and direct path to reconstruct the image.
 - However, since there is no randomness, the model cannot "correct" earlier mistakes effectively, which causes artifacts like the floating roof of the church becoming clouds, but still looking slightly off.
- **DDPM**, on the other hand, retains a positive noise variance $\sigma_t > 0$, adding stochastic noise at each step.
 - This randomness makes the sampling path non-deterministic, causing generated images at timesteps set to 10, to be much blurrier because the process is noisier and less direct.

- However, the noise term also allows the model to explore the latent space more flexibly and "correct" its trajectory over many steps, leading to significantly higher quality images when timesteps are set to 200.

Summary: The presence or absence of the noise term $\sigma_t \epsilon_t$ in the update formula is the reason for the trade-off between speed and correction ability: DDIM's deterministic path ($\sigma_t = 0$) enables faster but potentially less accurate reconstruction at low steps, while DDPM's stochastic path ($\sigma_t > 0$) yields slower but higher quality results due to better correction over many steps.

2 Task 2

I used default prompts for the generation, and they seemed fitting. There was no noticeable difference between images generated for the prompts "dog" and "sks dog." The animals were portrayed in various poses and environments. All images were of decent quality and portrayed the thing I asked for in the prompt.

Generated Images



Figure 5: a photo of sks dog



Figure 6: a photo of dog



Figure 7: a child playing

3 Task 3

After fine-tuning, images for the “sks dog” prompt became very similar to the target images. Unfortunately, the images generated for the “dog” prompt were very similar. The cat images were also altered: the cats were generated with a similar color palette to the “sks dog” and often had the same pose in the photo. I did not notice any noticeable differences in the sailboat and children photos.

Generated Images

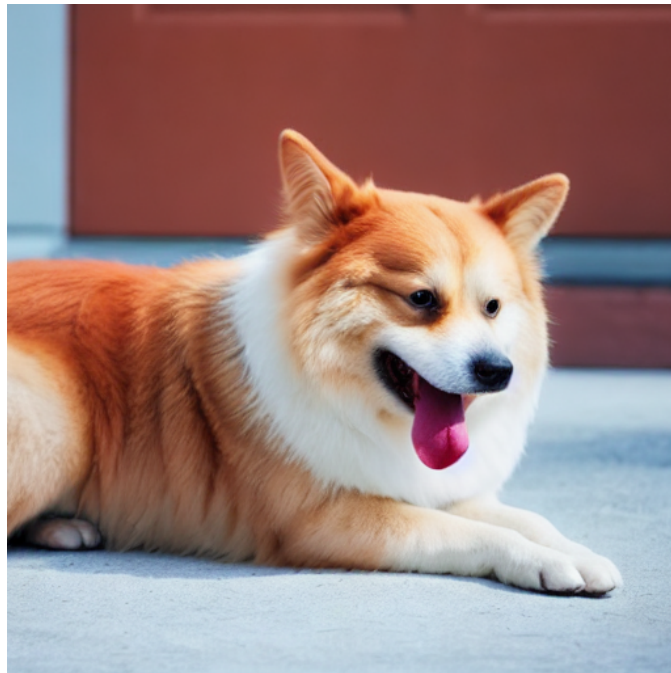


Figure 8: a photo of sks dog



Figure 9: a photo of a dog



Figure 10: a photo of a cat



Figure 11: a child playing

Findings

Without any additional tricks, we cannot expect the model to distinguish between the prompts “dog” and “sks dog”.

4 Task 4

Unfortunately, I did not know that I should generate images from the same noise, as this was not mentioned in the notebook. However, I believe my observations are quite universal. After training with additional dog images, the outputs for the “dog” prompt improved significantly. There was noticeable diversity in the pose and breed of the dogs, though the color palette remained similar. Images generated for the “sks dog” prompt were close to the target images, with the exception of one image. The images for the “cat” prompt also changed in color, but the poses of the cats remained very similar. Images of sailboats and children remained consistent with those generated by previous methods.

Generated Images



Figure 12: a photo of sks dog



Figure 13: a photo of sks dog



Figure 14: a photo of a dog



Figure 15: a photo of a cat



Figure 16: a child playing

Findings

Training with additional dog photos improved the diversity of generated images but lowered the overall robustness of the model.

5 Task 5

Overall, the higher the value of m , the worse the generated images become. I observed that using the original model as a suffix is more robust, even for higher values of m , while using it as a prefix results in poor images already at $m = 20$, with quality degrading further as m increases.

The suffix mode generates better images overall and starts producing bad images somewhere between $m = 20$ and $m = 30$. In my opinion, the best configuration is suffix mode with $m = 10$. For this configuration, the dog images are diverse and generally high quality. The "sks dog" images closely resemble the target images and look very good overall.

It makes sense that when m is around half of the total denoising steps (50), the images look bad, since we are not allowing either model to fully determine the image content, leading to inconsistent results. However, I find it difficult to explain why the images for $m = 40$ look bad in both suffix and prefix modes.

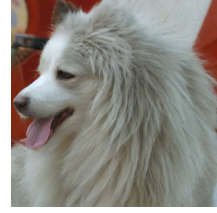
Generated Images



Dog, $m = 10$, prefix



Dog, $m = 10$, prefix



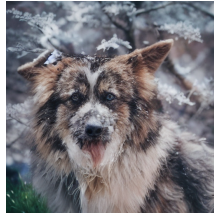
Dog, $m = 10$, suffix



Dog, $m = 10$, suffix



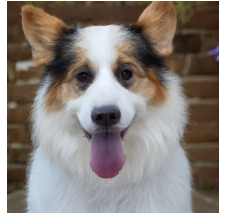
Dog, $m = 10$, suffix



Dog, $m = 20$, prefix



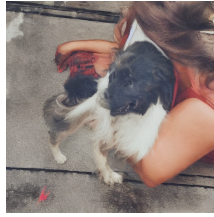
Dog, $m = 20$, prefix



Dog, $m = 20$, suffix



Dog, $m = 20$, suffix



Dog, $m = 30$, prefix



Dog, $m = 30$, prefix



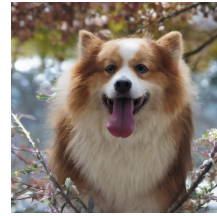
Dog, $m = 30$, suffix



Dog, $m = 30$, suffix



SKS Dog, $m = 10$, prefix



SKS Dog, $m = 10$, suffix



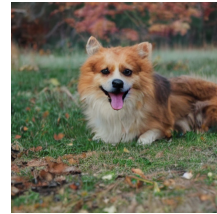
SKS Dog, $m = 20$, prefix



SKS Dog, $m = 20$, prefix



SKS Dog, $m = 20$, suffix



SKS Dog, $m = 20$, suffix



SKS Dog, $m = 30$, prefix

Figure 17: Samples of images generated using original and finetuned models.

Findings

The lower the value of m , the better the image quality. Among the configurations tested, using the suffix mode with $m = 10$ yields the best results.

Conclusion

Standard fine-tuning improves the generation quality for the target prompt but disrupts the model’s ability to generate other dogs and animals accurately. The use of prior preservation partially mitigates this issue by maintaining diversity for the general class, but it comes at the cost of decreased robustness when generating the images for target prompt. In my opinion, the best approach is the mixed denoising method, especially when using a low value of m combined with the suffix strategy. This configuration balances of the fine-tuned model while preserving generalization, resulting in improved image quality and diversity across prompts.